🌐 AWS Networking & VPC — Category 3: Complete Interview Guide

> **Target Audience:** DevOps / SRE / Platform / Cloud Engineers
> **Difficulty:** Beginner → Advanced
> **Topics Covered:** VPC Fundamentals, Security Groups, NACLs, VPC Peering, Transit Gateway, VPC Endpoints, Route 53, CloudFront, PrivateLink, Direct Connect, VPN, NAT Gateway, Egress-Only IGW

📋 Table of Contents

1. 3.1 VPC Fundamentals — Subnets, Route Tables, IGW, NAT
2. 3.2 Security Groups vs NACLs
3. 3.3 VPC Peering & Transit Gateway
4. 3.4 VPC Endpoints — Interface vs Gateway
5. 3.5 Route 53 — Hosted Zones, Routing Policies
6. 3.6 CloudFront — Distributions, Behaviors, Origins, Cache Invalidation
7. 3.7 PrivateLink
8. 3.8 Direct Connect & Site-to-Site VPN
9. 3.9 Network ACLs Deep Dive & Stateless vs Stateful
10. 3.10 Egress-Only IGW, NAT Gateway vs NAT Instance

3.1 VPC Fundamentals — Subnets, Route Tables, IGW, NAT

🟢 What It Is in Simple Terms

A VPC (Virtual Private Cloud) is your own private, isolated section of the AWS cloud — like having your own data center network inside AWS. You control the IP ranges, subnets, routing, and what can talk to what. Nothing gets in or out unless you explicitly allow it.

🔍 Why It Exists / What Problem It Solves

Before VPC, all EC2 instances shared a flat public network (EC2-Classic). Any instance could talk to any other instance. No isolation. No private networking. VPC solved this by giving every customer a logically isolated network they fully control — their own IP space, subnets, routing, and firewall rules.

⚙️ How It Works Internally

```
AWS Region: us-east-1
┌──────────────────────────────────────────────────────────────────┐
│  VPC: 10.0.0.0/16  (65,536 IP addresses)                         │
│                                                                   │
│  AZ: us-east-1a              AZ: us-east-1b                       │
│  ┌──────────────────┐        ┌──────────────────┐                │
│  │ Public Subnet    │        │ Public Subnet    │                │
│  │ 10.0.1.0/24      │        │ 10.0.2.0/24      │                │
│  │  [EC2] [ALB]     │        │  [EC2] [ALB]     │                │
│  └──────────────────┘        └──────────────────┘                │
│           │                          │                            │
│  ┌────────▼─────────┐        ┌────────▼─────────┐                │
│  │ Private Subnet   │        │ Private Subnet   │                │
│  │ 10.0.3.0/24      │        │ 10.0.4.0/24      │                │
│  │  [EC2] [RDS]     │        │  [EC2] [RDS]     │                │
│  └──────────────────┘        └──────────────────┘                │
│                                                                   │
│  Internet Gateway (IGW)  ←→  Public Internet                      │
│  NAT Gateway (in public subnet) ← private instances use this      │
└──────────────────────────────────────────────────────────────────┘
```

🧩 Key Components

CIDR Blocks & IP Addressing

```
VPC CIDR: 10.0.0.0/16
├── /16 = 65,536 total IPs
├── AWS reserves 5 IPs per subnet:
│   10.0.1.0   → Network address
│   10.0.1.1   → VPC router (default gateway)
│   10.0.1.2   → DNS server
│   10.0.1.3   → Reserved for future use
│   10.0.1.255 → Broadcast (not used, but reserved)
└── Usable IPs in a /24: 256 - 5 = 251

⚠ Gotcha: Smallest subnet AWS allows is /28 (11 usable IPs)
⚠ Gotcha: VPC CIDR cannot be changed after creation
           (you can ADD secondary CIDRs but not remove primary)

Subnet sizing guide:
/28 →   11 usable  (tiny, avoid)
/24 →  251 usable  (standard)
/22 → 1019 usable  (large)
/20 → 4091 usable  (very large)
```

Subnets — Public vs Private

```
What makes a subnet "public"?
A subnet is PUBLIC if:
1. It has a route to an Internet Gateway (0.0.0.0/0 → igw-xxx)
2. Instances have public IPs (auto-assign or Elastic IP)

A subnet is PRIVATE if:
1. No direct route to IGW
2. Outbound internet via NAT Gateway (or no internet at all)

⚠ Key misconception: "Public subnet" is NOT a property you set
on the subnet — it's purely defined by the route table.
A subnet with a route to an IGW = public subnet.

Subnet design best practice (3-tier):
┌──────────────────────────────────────────┐
│ Public Tier    (10.0.0.0/24 per AZ)      │ ← ALB, NAT GW, Bastion
│ Private Tier   (10.0.10.0/24 per AZ)     │ ← App servers, ECS, EKS
│ Database Tier  (10.0.20.0/24 per AZ)     │ ← RDS, ElastiCache
└──────────────────────────────────────────┘
```

Route Tables

```
Route tables control WHERE traffic is sent.
Every subnet must be associated with exactly one route table.
(Multiple subnets can share one route table)

Public subnet route table:
┌──────────────────┬──────────────────────┐
│ Destination      │ Target               │
├──────────────────┼──────────────────────┤
│ 10.0.0.0/16      │ local                │ ← VPC-internal traffic
│ 0.0.0.0/0        │ igw-0abc123          │ ← Internet traffic → IGW
└──────────────────┴──────────────────────┘

Private subnet route table:
┌──────────────────┬──────────────────────┐
│ Destination      │ Target               │
├──────────────────┼──────────────────────┤
│ 10.0.0.0/16      │ local                │ ← VPC-internal traffic
│ 0.0.0.0/0        │ nat-0xyz789          │ ← Internet traffic → NAT GW
└──────────────────┴──────────────────────┘

Database subnet route table (no internet):
┌──────────────────┬──────────────────────┐
│ Destination      │ Target               │
├──────────────────┼──────────────────────┤
│ 10.0.0.0/16      │ local                │ ← VPC-internal only
└──────────────────┴──────────────────────┘
```

```
Route priority: Most specific route wins
10.0.1.5/32 > 10.0.1.0/24 > 10.0.0.0/16 > 0.0.0.0/0

Example: VPC Endpoint affecting routing

Private subnet route table after adding S3 Gateway Endpoint:

┌──────────────────────────────┬──────────────────────┐
│ Destination                  │ Target               │
├──────────────────────────────┼──────────────────────┤
│ 10.0.0.0/16                  │ local                │
│ pl-63a5400a (S3 prefix list)│ VPC Endpoint         │ ← S3 traffic
│ 0.0.0.0/0                    │ nat-0xyz789          │ ← Other internet
traffic
└──────────────────────────────┴──────────────────────┘

Result:
EC2 → S3 traffic uses VPC Endpoint
NOT → NAT Gateway

Reason:
The S3 prefix list is more specific than 0.0.0.0/0.

Implications:
• S3 traffic stays inside AWS network
• NAT Gateway is bypassed
• Reduces NAT data processing cost
• Improves security (no public internet path)
```

Internet Gateway (IGW)

```
What IGW does:
├── Allows communication between VPC and the internet
├── Horizontally scaled, redundant, HA — no bandwidth limit
├── Performs NAT for instances with public IPs
│   (translates public IP ↔ private IP)
└── One IGW per VPC (but a VPC can detach/reattach)

IGW is stateful NAT:
Instance (10.0.1.5) sends to 8.8.8.8
→ IGW translates source: 10.0.1.5 → 54.12.34.56 (Elastic IP)
→ Response comes back to 54.12.34.56
→ IGW translates back to 10.0.1.5

Without a public IP on the instance:
→ IGW cannot translate → no internet access
  even with a route to IGW
```

NAT Gateway

```
Purpose: Allow PRIVATE subnet instances to initiate
         outbound internet connections (e.g., yum update)
         WITHOUT being reachable from the internet.

NAT Gateway properties:
├── Deployed IN a public subnet
├── Has an Elastic IP
├── Managed by AWS — no patching needed
├── Scales automatically up to 45 Gbps
├── Per-AZ — you need one NAT GW per AZ for HA
└── Costs money: ~$0.045/hr + $0.045/GB data processed

⚠ Critical: NAT Gateway is AZ-specific
If AZ-a's NAT GW fails and private subnet in AZ-b
routes through it, AZ-b loses internet too!

Correct HA pattern (one NAT GW per AZ):

AZ-a:                           AZ-b:
Public Subnet                   Public Subnet
[NAT GW A] ←── Private         [NAT GW B] ←── Private
                subnet AZ-a                     subnet AZ-b

Each private subnet routes to the NAT GW in its OWN AZ.

NAT Gateway traffic flow:
Private EC2 (10.0.3.5)
  → Route: 0.0.0.0/0 → nat-gateway
  → NAT GW translates to its Elastic IP
  → Traffic exits via IGW
  → Response returns to NAT GW EIP
  → NAT GW translates back to 10.0.3.5
  → EC2 gets response

Single NAT Gateway is both a reliability risk AND a cost trap — cross-AZ
traffic to reach a single NAT Gateway adds $0.01/GB inter-AZ charge on top
of the $0.045/GB NAT processing charge. At meaningful traffic volumes,
three NAT Gateways (one per AZ) costs less than one NAT Gateway with
cross-AZ traffic.

Always route each private subnet to its AZ-local NAT Gateway — this is
both the HA best practice and the cost-optimal pattern.
```

AWS CLI — VPC Setup

```
# Create VPC
aws ec2 create-vpc --cidr-block 10.0.0.0/16 \
```

```
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=prod-vpc}]'

# Create public subnet
aws ec2 create-subnet \
  --vpc-id vpc-0abc123 \
  --cidr-block 10.0.1.0/24 \
  --availability-zone us-east-1a

# Create Internet Gateway and attach
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway \
  --internet-gateway-id igw-0abc123 \
  --vpc-id vpc-0abc123

# Create route table for public subnet
aws ec2 create-route-table --vpc-id vpc-0abc123
aws ec2 create-route \
  --route-table-id rtb-0abc123 \
  --destination-cidr-block 0.0.0.0/0 \
  --gateway-id igw-0abc123

# Associate route table with subnet
aws ec2 associate-route-table \
  --route-table-id rtb-0abc123 \
  --subnet-id subnet-0abc123

# Create NAT Gateway (needs EIP first)
aws ec2 allocate-address --domain vpc
aws ec2 create-nat-gateway \
  --subnet-id subnet-0abc123 \
  --allocation-id eipalloc-0abc123
```

💬 Short Crisp Interview Answer

"A VPC is your isolated private network in AWS. You define a CIDR range, divide it into subnets across AZs, and control routing with route tables. A subnet becomes public by having a route to an Internet Gateway — that's it, it's a routing concept, not a subnet property. Private subnets route outbound traffic through a NAT Gateway, which lives in a public subnet and has an Elastic IP. For high availability, you deploy one NAT Gateway per AZ so a single AZ failure doesn't take down internet access for other AZs."

🔬 Deep Dive Version

"VPC CIDR blocks can't be changed after creation, but you can add secondary CIDRs. AWS reserves 5 IPs per subnet. Route tables use longest-prefix matching — the most specific route wins. The IGW performs stateful NAT between public IPs and private IPs on instances. NAT Gateway is AZ-scoped, managed, scales to 45 Gbps, but costs per-hour and per-GB — a common hidden cost. The correct HA pattern is one NAT Gateway per AZ with each private subnet routing to its local NAT Gateway. A common architecture mistake is sharing one NAT Gateway across AZs — this creates both a single point of failure and inter-AZ data transfer costs."

🏭 Real World Production Example

```
Production 3-tier VPC for a SaaS platform (us-east-1):

VPC: 10.0.0.0/16

AZ-a (us-east-1a)          AZ-b (us-east-1b)         AZ-c (us-east-1c)
──────────────────         ─────────────────          ─────────────────
Public: 10.0.1.0/24        Public: 10.0.2.0/24        Public: 10.0.3.0/24
  [ALB node]                 [ALB node]                 [ALB node]
  [NAT GW A]                 [NAT GW B]                 [NAT GW C]

App:  10.0.11.0/24          App:  10.0.12.0/24         App:  10.0.13.0/24
  [ECS tasks]                 [ECS tasks]                [ECS tasks]
  → route: 0/0 → NAT-A       → route: 0/0 → NAT-B      → route: 0/0 → NAT-
C

DB:   10.0.21.0/24          DB:   10.0.22.0/24         DB:   10.0.23.0/24
  [RDS primary]               [RDS standby]              [RDS replica]
  NO internet route           NO internet route          NO internet route

Result:
- Single AZ failure loses only that AZ's resources
- No cross-AZ NAT traffic (cost savings)
- DB tier completely isolated from internet
- ~$100/month for 3 NAT Gateways (justified for prod HA)
```

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
VPC CIDR immutable	Can't change primary CIDR. Plan upfront. Use /16 for flexibility
NAT GW is AZ-specific	One per AZ for HA. Single NAT GW = single point of failure
IGW requires public IP	Instance needs a public IP AND route to IGW. Either alone is insufficient
Route table default	Main route table has only the `local` route. You must add the IGW route manually
Subnet association	A subnet with no explicit route table association uses the main route table
5 reserved IPs	Always subtract 5 from subnet CIDR for usable IP count
NAT GW vs NAT Instance	NAT GW is managed, scales, costs more. NAT Instance is DIY on EC2, cheaper, but you manage HA

🔗 Connections to Other AWS Concepts

```
VPC connects to:
├── EC2          → Instances live in subnets
├── RDS          → DB subnet groups span multiple AZs
├── ECS/EKS      → Container networking in VPC
├── ALB/NLB      → Load balancers span public subnets
├── Security Groups → Instance-level firewall
├── NACLs        → Subnet-level firewall
├── VPC Peering  → Connect two VPCs
├── Transit GW   → Hub-and-spoke multi-VPC routing
├── VPC Endpoints → Private AWS service access
├── PrivateLink  → Expose services privately
├── Flow Logs    → Network traffic logging
└── Direct Connect → On-prem to VPC private link
```

3.2 Security Groups vs NACLs

🟢 What It Is in Simple Terms

Two layers of network firewall in AWS. Security Groups protect individual resources (like EC2 instances). NACLs (Network Access Control Lists) protect entire subnets. They work together as defense in depth.

⚙️ How They Work — Traffic Flow

```
Traffic flow through both layers:

Internet
   │
   ▼
[NACL] ← subnet boundary check (inbound)
   │
   ▼
[Security Group] ← instance boundary check (inbound)
   │
   ▼
[EC2 Instance]
   │
   ▼
[Security Group] ← instance boundary check (outbound)
   │
   ▼
[NACL] ← subnet boundary check (outbound)
   │
```

```
    ▼
Internet
```

🧩 Deep Comparison

```
┌──────────────────────┬──────────────────────┬──────────────────────┐
│ Feature              │ Security Group       │ NACL                 │
├──────────────────────┼──────────────────────┼──────────────────────┤
│ Applies to           │ Instance/ENI level   │ Subnet level         │
│ State                │ STATEFUL             │ STATELESS            │
│ Rules                │ Allow only           │ Allow AND Deny       │
│ Rule evaluation      │ All rules evaluated  │ Rules evaluated in   │
│                      │ together             │ NUMBER ORDER (stops  │
│                      │                      │ at first match)      │
│ Inbound/Outbound     │ Separate rules but   │ Completely separate  │
│                      │ return traffic auto  │ — must allow both    │
│                      │ allowed (stateful)   │ directions manually  │
│ Default (new VPC)    │ Deny all inbound     │ Allow all            │
│                      │ Allow all outbound   │ (rule 100 allow all) │
│ Default (custom)     │ Deny all in/out      │ Deny all             │
└──────────────────────┴──────────────────────┴──────────────────────┘
```

Security Groups — Deep Dive

```
Key properties:
├── STATEFUL: If you allow inbound port 80, the response is
│            automatically allowed outbound. You don't need
│            an explicit outbound rule for return traffic.
├── Allow rules ONLY: You cannot write a "deny" rule in an SG
├── Applied to ENI: Technically applied to the Elastic Network
│   Interface, not the instance itself
└── Can reference other SGs (not just IPs)
```

SG referencing other SGs — the cleanest pattern:

```
┌──────────────────────────────────────────────────────────┐
│  ALB Security Group (sg-alb)                             │
│  Inbound: 0.0.0.0/0 port 443                             │
└──────────────────────────────────────────────────────────┘
                     │ sends traffic to
┌──────────────────────────────────────────────────────────┐
│  App Security Group (sg-app)                             │
│  Inbound: source=sg-alb port 8080                        │ ← Only ALB can
reach app
└──────────────────────────────────────────────────────────┘
```

```
                    │ sends traffic to
┌──────────────────────────────────────────────────────────┐
│  DB Security Group (sg-db)                               │
│  Inbound: source=sg-app port 5432                        │ ← Only app can
reach DB
└──────────────────────────────────────────────────────────┘

This pattern is far better than IP-based rules because:
- Auto-adjusts as new instances launch (dynamic IPs)
- Zero maintenance as fleet scales
- Works across ASG, ECS, EKS
```

```
# Create security group
aws ec2 create-security-group \
  --group-name app-sg \
  --description "App tier security group" \
  --vpc-id vpc-0abc123

# Allow inbound from ALB SG only (SG-to-SG reference)
aws ec2 authorize-security-group-ingress \
  --group-id sg-app123 \
  --protocol tcp \
  --port 8080 \
  --source-group sg-alb456

# Allow outbound to DB SG only
aws ec2 authorize-security-group-egress \
  --group-id sg-app123 \
  --protocol tcp \
  --port 5432 \
  --destination-group sg-db789
```

NACLs — Deep Dive

```
Key properties:
├── STATELESS: Return traffic must be explicitly allowed.
│            If you allow inbound port 80, you MUST also
│            allow outbound ephemeral ports for the response.
├── Allow AND Deny rules: You CAN block specific IPs
├── Rule number order: Rules evaluated lowest number first.
│   FIRST MATCH WINS — processing stops.
└── Subnet-level: Applies to ALL traffic entering/leaving subnet

Ephemeral ports (⚠ critical for NACL):
When a client connects to port 80 on your server,
the client uses a random "ephemeral" source port: 1024-65535

NACL inbound rules must allow:  port 80 (destination)
```

```
NACL outbound rules must allow: ports 1024-65535 (ephemeral)
Without the outbound ephemeral rule → client never gets response!

Example NACL for public subnet:

INBOUND:
┌──────┬──────────┬──────────┬──────────┬──────┐
│ Rule │ Protocol │ Port Range │ Source   │ Action │
├──────┼──────────┼──────────┼──────────┼──────┤
│ 100  │ TCP      │ 443      │ 0.0.0.0/0 │ ALLOW │
│ 110  │ TCP      │ 80       │ 0.0.0.0/0 │ ALLOW │
│ 120  │ TCP      │ 1024-65535 │ 0.0.0.0/0 │ ALLOW │ ← ephemeral (return)
│ *    │ ALL      │ ALL      │ 0.0.0.0/0 │ DENY  │ ← implicit deny all
└──────┴──────────┴──────────┴──────────┴──────┘

OUTBOUND:
┌──────┬──────────┬──────────┬──────────┬──────┐
│ Rule │ Protocol │ Port Range │ Dest     │ Action │
├──────┼──────────┼──────────┼──────────┼──────┤
│ 100  │ TCP      │ 443      │ 0.0.0.0/0 │ ALLOW │
│ 110  │ TCP      │ 80       │ 0.0.0.0/0 │ ALLOW │
│ 120  │ TCP      │ 1024-65535 │ 0.0.0.0/0 │ ALLOW │ ← response traffic
│ *    │ ALL      │ ALL      │ 0.0.0.0/0 │ DENY  │
└──────┴──────────┴──────────┴──────────┴──────┘

When to actually use NACLs:
├── Blocking a known bad IP at subnet level (DDoS mitigation)
├── Compliance requirement: "subnet-level firewall must exist"
├── Extra defense layer for sensitive subnets (DB tier)
└── Blocking traffic between subnets in same VPC
```

💬 Short Crisp Interview Answer

"Security Groups and NACLs are two layers of firewall in AWS. Security Groups operate at the instance/ENI level, are stateful — meaning return traffic is automatically allowed — and support allow rules only. NACLs operate at the subnet level, are stateless — you must explicitly allow both inbound and outbound including ephemeral ports — and support both allow and deny rules. In practice, Security Groups do most of the work. NACLs are used for subnet-level blocking, like denying a bad IP range during a DDoS attack. The key gotcha on NACLs: because they're stateless, forgetting outbound ephemeral port rules will silently break connectivity."

🏭 Real World Production Example

```
DDoS Mitigation using NACLs:

10:30 AM: Security team detects attack from 1.2.3.0/24
          flooding app servers with SYN packets
```

```
Immediate response:
aws ec2 create-network-acl-entry \
  --network-acl-id acl-public \
  --rule-number 50 \           ← Low number = high priority
  --protocol tcp \
  --rule-action deny \
  --ingress \
  --cidr-block 1.2.3.0/24 \
  --port-range From=0,To=65535

Effect: ALL traffic from 1.2.3.0/24 blocked at subnet boundary
        before it even reaches EC2 instances or Security Groups.
        NACL deny rule 50 evaluated before allow rule 100.

This is something you CANNOT do with Security Groups alone
(SGs only have allow rules).
```

⚠️ Tricky Edge Cases / Gotchas

Gotcha	Detail
NACL stateless + ephemeral ports	Must allow outbound 1024-65535 for response traffic
NACL rule order matters	Rule 100 allow + rule 200 deny on same IP = ALLOW wins (first match stops processing)
SG default outbound	New SGs allow ALL outbound by default. Lock this down in prod
SG changes are instant	No need to restart instances — takes effect immediately
NACL applies to ALL subnet traffic	Including traffic between subnets in same VPC
SG max rules	60 inbound + 60 outbound per SG. 5 SGs per ENI

3.3 VPC Peering & Transit Gateway

🟢 What It Is in Simple Terms

VPC Peering lets two VPCs talk to each other as if they were on the same network. Transit Gateway is a hub-and-spoke router that lets many VPCs and on-prem networks connect through one central point — instead of a web of peer-to-peer connections.

⚙️ VPC Peering — How It Works

```
VPC Peering:

VPC-A (10.0.0.0/16) ←──peering──→ VPC-B (172.16.0.0/16)

Requirements:
├── No overlapping CIDR blocks (critical!)
├── Peering connection must be accepted by the other side
├── Route tables in BOTH VPCs must be updated
├── Security Groups must allow traffic
└── Works cross-account and cross-region

Route table update required (both sides):

VPC-A route table:

┌──────────────────┬──────────────────────────┐
│ 10.0.0.0/16      │ local                    │
│ 172.16.0.0/16    │ pcx-0abc123 (peering)    │ ← Added manually
└──────────────────┴──────────────────────────┘

VPC-B route table:

┌──────────────────┬──────────────────────────┐
│ 172.16.0.0/16    │ local                    │
│ 10.0.0.0/16      │ pcx-0abc123 (peering)    │ ← Added manually
└──────────────────┴──────────────────────────┘
```

⚠ VPC Peering Limitations — Non-Transitivity

```
CRITICAL: VPC Peering is NON-TRANSITIVE

VPC-A ←── peered ──→ VPC-B ←── peered ──→ VPC-C

VPC-A CANNOT talk to VPC-C through VPC-B!
Each pair needs its own peering connection.

With 10 VPCs fully meshed:
Peering connections needed = n(n-1)/2 = 10×9/2 = 45 connections
With 100 VPCs = 4,950 connections!

This is why Transit Gateway exists.
```

```
# Create VPC peering
aws ec2 create-vpc-peering-connection \
  --vpc-id vpc-a \
  --peer-vpc-id vpc-b \
  --peer-owner-id 123456789012 \   # cross-account
  --peer-region us-west-2           # cross-region
```

```
# Accept peering (from other account/region)
aws ec2 accept-vpc-peering-connection \
  --vpc-peering-connection-id pcx-0abc123

# Add route in VPC-A
aws ec2 create-route \
  --route-table-id rtb-a \
  --destination-cidr-block 172.16.0.0/16 \
  --vpc-peering-connection-id pcx-0abc123
```

⚙️ Transit Gateway — How It Works

```
Transit Gateway (TGW):
Hub-and-spoke model — all VPCs connect to the TGW,
which routes traffic between them.

Without TGW (meshed peering — 6 VPCs = 15 connections):

VPC1 ——— VPC2
 |  \    /  |
 |   \  /   |
VPC3 —— VPC4
 |  \    /  |
 |   \  /   |
VPC5 ——— VPC6

With TGW (all connect to hub):

VPC1    VPC2    VPC3
  \      |      /
   \     |     /
    [Transit Gateway]
   /     |     \
  /      |      \
VPC4   VPC5   On-Prem (via VPN/Direct Connect)

TGW components:
├── Attachments: Each VPC/VPN/DX attaches to TGW
├── Route tables: TGW has its own route tables
│   (separate from VPC route tables)
├── Propagations: Routes auto-propagated from attachments
└── Associations: Each attachment associates with one TGW RT

TGW routing (default — full mesh):
All VPCs attached → all can reach all others

TGW routing (segmented — isolated):
Dev VPCs  → TGW route table A (no prod routes)
Prod VPCs → TGW route table B (no dev routes)
Shared services VPC → propagated to ALL route tables
```

TGW vs VPC Peering — Decision Matrix

```
┌──────────────────────┬──────────────────┬──────────────────┐
│ Factor               │ VPC Peering      │ Transit GW       │
├──────────────────────┼──────────────────┼──────────────────┤
│ Cost                 │ Free (data       │ $0.05/hr/attach  │
│                      │ transfer only)   │ + data transfer  │
│ Scale                │ ≤10 VPCs         │ 1000s of VPCs    │
│ Transitivity         │ ❌ No            │ ✅ Yes           │
│ On-prem connectivity │ ❌ No            │ ✅ Via VPN/DX    │
│ Multicast            │ ❌ No            │ ✅ Yes           │
│ Bandwidth            │ No limit         │ 50 Gbps/attach   │
│ Cross-account        │ ✅ Yes           │ ✅ Yes (RAM)     │
│ Cross-region         │ ✅ Yes           │ ✅ Peering TGWs  │
│ Latency              │ Lowest           │ Slightly higher  │
└──────────────────────┴──────────────────┴──────────────────┘

Rule of thumb:
≤3 VPCs = Peering (free, simple)
>3 VPCs or need on-prem = Transit Gateway
```

💬 Short Crisp Interview Answer

"VPC Peering creates a direct one-to-one connection between two VPCs — it requires no overlapping CIDRs, manual route table updates on both sides, and is non-transitive, meaning A-to-B and B-to-C peering doesn't let A reach C. Transit Gateway solves this by acting as a hub-and-spoke router — all VPCs attach to it and it routes between them. It also connects to on-prem via VPN or Direct Connect. For small environments with 2-3 VPCs, peering is simpler and free. Beyond that, or when you need on-prem connectivity or network segmentation between prod and dev, Transit Gateway is the right choice."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Peering is non-transitive	A→B and B→C does NOT mean A→C. This is the #1 gotcha
Overlapping CIDRs	Peering fails if VPC CIDRs overlap. Plan CIDR strategy upfront
Route tables not auto-updated	Peering connection doesn't add routes. You must add them manually to both sides
TGW costs add up	$0.05/hr per attachment × 50 VPCs = $1,800/month just in attachment fees
TGW cross-region	Requires TGW peering (TGW to TGW), not direct VPC attachment

- **TGW supports overlapping CIDRs across attachments** — VPC Peering does not — this is a fundamental architectural difference. When spoke VPCs have overlapping CIDRs, TGW is the only viable hub-and-spoke option.
- **Shared Services cannot initiate connections to overlapping-CIDR spokes** — only the spoke→SharedSvc direction is unambiguous with overlapping CIDRs on TGW. For management traffic requiring the reverse direction, use SSM agent outbound calls instead.
- **TGW route table association vs propagation** — each attachment is associated with ONE route table (controls routing FROM that attachment) but can propagate routes INTO multiple route tables. Candidates frequently confuse these two concepts.

3.4 VPC Endpoints — Interface vs Gateway

🟢 What It Is in Simple Terms

VPC Endpoints let your VPC resources reach AWS services like S3, DynamoDB, or any AWS service **without going through the internet**. Traffic stays on AWS's private network. This is faster, cheaper, and more secure.

⚙️ How It Works

```
Without VPC Endpoint (traffic goes to internet):

Private EC2 ──→ NAT GW ──→ IGW ──→ Internet ──→ S3
Cost: NAT GW data fee + data transfer fee
Security: Traffic briefly on public internet

With VPC Endpoint (traffic stays private):

Private EC2 ──→ VPC Endpoint ──→ S3 (AWS backbone)
Cost: No NAT GW fee, no data transfer fee
Security: Traffic never leaves AWS network
```

🧩 Two Types of VPC Endpoints

Gateway Endpoints (Free!)

```
Supports: S3 and DynamoDB ONLY
Cost: FREE
How it works: Adds a route to your route table
              (no ENI, no DNS change)

Route table after gateway endpoint:
```

```
  ┌──────────────────────┬──────────────────────────┐
  │ 10.0.0.0/16          │ local                    │
  │ 0.0.0.0/0            │ nat-gateway              │
  │ pl-68a54001 (S3)     │ vpce-0abc123             │ ← Added by endpoint
  └──────────────────────┴──────────────────────────┘

  pl-68a54001 = AWS prefix list for S3 IPs in this region
  Traffic to any S3 IP → goes to endpoint, not NAT GW

  ⚠ Gateway endpoints are region-specific.
     Can't use us-east-1 S3 endpoint to reach us-west-2 S3.

  Endpoint policy (restrict what can be accessed):
  {
    "Statement": [{
      "Effect": "Allow",
      "Principal": "*",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::my-company-bucket/*"
    }]
  }
  This prevents instances from accessing OTHER S3 buckets
  (e.g., data exfiltration to external S3).
```

```
# Create S3 Gateway Endpoint
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-0abc123 \
  --service-name com.amazonaws.us-east-1.s3 \
  --vpc-endpoint-type Gateway \
  --route-table-ids rtb-private1 rtb-private2
```

Interface Endpoints (AWS PrivateLink)

```
  Supports: 100+ AWS services (SSM, ECR, KMS, CloudWatch,
            Secrets Manager, STS, API Gateway, etc.)
  Cost: ~$0.01/hr per AZ + $0.01/GB data
  How it works: Creates an ENI in your subnet with a private IP.
                DNS resolves service hostname to private IP.

  Without interface endpoint:
  ec2.us-east-1.amazonaws.com → resolves to public IP
  → traffic goes via internet (or NAT GW)

  With interface endpoint:
  ec2.us-east-1.amazonaws.com → resolves to 10.0.3.x (ENI IP)
  → traffic stays in VPC, goes via AWS backbone
  (DNS resolution handled automatically via Route 53 Private Hosted Zone)
```

```
Interface Endpoint Architecture:

┌──────────────────────────────────────────────┐
│  VPC (10.0.0.0/16)                           │
│                                              │
│  Private Subnet A          Private Subnet B  │
│   ┌──────────┐              ┌──────────┐     │
│   │ ENI      │              │ ENI      │     │
│   │10.0.3.5  │              │10.0.4.5  │     │
│   │(endpoint)│              │(endpoint)│     │
│   └──────────┘              └──────────┘     │
│        │                        │            │
└────────┼────────────────────────┼────────────┘
         │ AWS PrivateLink        │
         ▼                        ▼
    AWS Service (SSM, ECR, CloudWatch, etc.)
```

```
# Create SSM Interface Endpoint
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-0abc123 \
  --vpc-endpoint-type Interface \
  --service-name com.amazonaws.us-east-1.ssm \
  --subnet-ids subnet-private-a subnet-private-b \
  --security-group-ids sg-endpoint \
  --private-dns-enabled   # ← DNS auto-resolved to private IP
```

Gateway vs Interface — When to Use

```
┌──────────────────┬──────────────┬──────────────────────┐
│ Feature          │ Gateway EP   │ Interface EP         │
├──────────────────┼──────────────┼──────────────────────┤
│ Services         │ S3, DynamoDB │ 100+ services        │
│ Cost             │ FREE         │ ~$7/month/AZ         │
│ How it works     │ Route entry  │ ENI in your subnet   │
│ Accessible from  │ Same VPC only│ On-prem (via DX/VPN) ✅ │
│ Security         │ Endpoint policy│ Endpoint policy + SG │
└──────────────────┴──────────────┴──────────────────────┘

Key advantage of Interface EP over Gateway EP:
Interface endpoints work from on-prem (via Direct Connect/VPN).
Gateway endpoints ONLY work from within the VPC.
If your on-prem servers need to access S3 privately → Interface EP.
```

💬 Short Crisp Interview Answer

"VPC Endpoints allow private communication to AWS services without going through the internet. There are two types. Gateway Endpoints are free and work for S3 and DynamoDB only — they add a route to your

route table. Interface Endpoints use AWS PrivateLink — they create an ENI in your subnet and use private DNS so traffic never leaves the AWS network. Interface endpoints cost money but support 100+ services and work from on-premises via Direct Connect or VPN, which Gateway Endpoints don't. A common production use case: using an S3 Gateway Endpoint to avoid NAT Gateway data processing fees on high-volume S3 traffic."

🏭 Real World Production Example

```
Cost optimization at a data company:

Problem: 50TB/day flowing from EC2 → NAT GW → S3
         NAT GW costs: 50TB × $0.045/GB = $2,250/DAY

Solution: S3 Gateway Endpoint (FREE)
- Add gateway endpoint to all private subnet route tables
- S3 traffic routes directly via endpoint
- Zero NAT GW data processing fee

Savings: $2,250/day = $67,500/month
Endpoint cost: $0

Additionally: Add Interface endpoints for SSM, ECR, CloudWatch
- Eliminate internet dependency for these services
- EC2 instances in private subnet no longer need NAT GW
  for AWS API calls
- Can remove NAT GW from some environments entirely
```

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Gateway EP is route-based	Only works within VPC — on-prem traffic can't use it
Interface EP DNS	Must enable `privateDnsEnabled`. Without it, public DNS still resolves to public IPs
Endpoint policies	Separate from S3 bucket policies. Both must allow. Endpoint policy = "can this VPC access this resource?"
Interface EP SG	The endpoint ENI needs a Security Group — allow HTTPS (443) from your resources
Gateway EP regional	S3 gateway endpoint only routes to S3 in the SAME region

---

3.5 Route 53 — Hosted Zones, Routing Policies

🟢 What It Is in Simple Terms

Route 53 is AWS's DNS service. It translates domain names like `api.myapp.com` into IP addresses. But it's much more than basic DNS — it has health checks and routing policies that make it a powerful global load balancer and failover tool.

⚙️ How It Works Internally

```
DNS Resolution Flow:
User types: api.myapp.com

1. Browser checks local cache
2. OS checks /etc/hosts
3. Query goes to Recursive resolver (ISP/8.8.8.8)
4. Recursive resolver asks Root nameserver (.)
5. Root server returns TLD nameserver (.com)
6. TLD server returns authoritative nameserver (Route 53)
7. Route 53 returns the answer: 54.12.34.56
8. Browser connects to 54.12.34.56

Route 53 is the authoritative nameserver for your domain.
```

🧩 Hosted Zones

```
Public Hosted Zone:
├── Routes internet traffic to your resources
├── Visible to the entire internet
├── Cost: $0.50/month per zone
└── Example: myapp.com → ALB public DNS

Private Hosted Zone:
├── Routes traffic WITHIN a VPC (or multiple VPCs)
├── Not visible outside the VPC
├── Perfect for internal service discovery
├── Must associate with one or more VPCs
└── Example: api.internal → 10.0.1.5

Split-horizon DNS:
Same domain resolves differently inside and outside VPC:
Inside VPC:  api.myapp.com → 10.0.1.5 (private IP, private HZ)
Outside VPC: api.myapp.com → 54.12.34.56 (public IP, public HZ)
```

🧩 Record Types

```
A     → IPv4 address         (api.myapp.com → 1.2.3.4)
AAAA  → IPv6 address         (api.myapp.com → 2001:db8::1)
CNAME → Canonical name alias (www → myapp.com)
       ⚠ CANNOT be used at zone apex (root domain)
ALIAS → AWS extension of A   (myapp.com → ALB DNS name)
       ✅  CAN be used at zone apex
       Free queries (unlike CNAME which costs extra hops)
MX    → Mail server
TXT   → Text (used for verification, SPF, DKIM)
NS    → Name servers
SOA   → Start of Authority

⚠ ALIAS vs CNAME critical difference:
CNAME: myapp.com → otherdomain.com (not allowed at apex)
ALIAS: myapp.com → alb-123.us-east-1.elb.amazonaws.com ✅
       Alias resolves within Route 53, no extra DNS hop.
       No charge for alias queries to AWS resources.
```

🧩 Routing Policies ⚠ (Most Interviewed Topic)

1. Simple Routing

```
One record → one or more IPs (returns all, client picks randomly)
No health checks
Use: Single resource, no failover needed
```

2. Weighted Routing

```
Split traffic by percentage across multiple resources.
Weight 70 → Resource A, Weight 30 → Resource B

Use cases:
├── Blue/green deployments (shift traffic gradually)
├── A/B testing (10% to new version)
└── Gradual region migration

Weight 0 = no traffic (exclude without deleting record)
```

3. Latency-Based Routing

```
Routes user to the AWS region with LOWEST LATENCY
(not necessarily geographically closest)
```

```
Must create records in each region.
Use: Global multi-region deployments.

⚠ Latency ≠ Geography. AWS measures actual latency,
which can differ from proximity.
```

4. Failover Routing

```
Primary → Secondary (only when primary health check fails)
Active-passive HA pattern.

Primary:   api.myapp.com → us-east-1 ALB (health check required)
Secondary: api.myapp.com → us-west-2 ALB (standby)

When primary health check fails → all traffic to secondary.
When primary recovers → traffic returns to primary.
```

5. Geolocation Routing

```
Routes based on USER'S GEOGRAPHIC LOCATION
(country, continent, or US state level)

Germany  → eu-west-1 (GDPR compliance)
USA      → us-east-1
Default  → us-east-1 (must have default or unmatched = no answer)

Use cases:
├── Data sovereignty / compliance
├── Localized content
└── Restrict access by country
```

6. Geoproximity Routing (Traffic Flow only)

```
Route based on geographic location of resources AND users.
Can BIAS routing (expand or shrink a resource's coverage).
Bias +50 → resource attracts MORE traffic from wider area.
Bias -50 → resource attracts LESS traffic.
Use: Fine-tune geographic distribution.
```

7. Multivalue Answer Routing

```
Returns up to 8 healthy records, client picks one.
Similar to Simple but with health checks per record.
```

```
⚠ NOT a replacement for a load balancer.
Use: Simple client-side load balancing.
```

Health Checks

```
Route 53 health checkers (from 15+ global locations):

Types:
├── Endpoint health check → HTTP/HTTPS/TCP to your resource
├── Calculated health check → AND/OR of multiple checks
└── CloudWatch alarm check → Based on CW metric

Health check options:
- Protocol: HTTP, HTTPS, TCP
- Path: /health
- Interval: 10s (fast) or 30s (standard)
- Threshold: 3 failures = unhealthy
- String matching: Look for specific string in response body

Calculated health check pattern:
┌──────────────────────────────────────────┐
│ Parent: "Is my service healthy overall?"  │
│ ├── Child 1: API health check             │
│ ├── Child 2: Database reachability        │
│ └── Child 3: Cache availability           │
│ Rule: 2 of 3 must be healthy              │
└──────────────────────────────────────────┘
Failover triggers only if 2+ components are unhealthy.
```

💬 Short Crisp Interview Answer

"Route 53 is AWS's authoritative DNS service. It supports public and private hosted zones — private zones enable split-horizon DNS where the same domain resolves to different IPs inside and outside a VPC. The key differentiator is its routing policies: simple for basic DNS, weighted for gradual traffic shifting and A/B testing, latency-based for global routing to the lowest-latency region, failover for active-passive HA, and geolocation for compliance and data sovereignty. Route 53 health checks integrate with failover policies to automatically route around failures. One critical distinction: use ALIAS records instead of CNAME at the zone apex — CNAME isn't allowed there, and ALIAS resolves within Route 53 at no extra charge."

🏭 Real World Production Example

```
Global SaaS product — multi-region active-active:

                    api.myapp.com
                         │
```

```
                    Route 53
              ┌──────────┼──────────┐
          Latency      Latency     Latency
              │          │          │
          us-east-1    eu-west-1  ap-south-1
          [ALB+ECS]    [ALB+ECS]  [ALB+ECS]
          HC: /health  HC: /health HC: /health

  Routing: Latency-based → user gets lowest latency region.
           Health checks → if a region's /health fails,
             Route 53 stops routing there automatically.

  Gradual rollout of new version:
  Week 1: Weighted 95/5 → old/new
  Week 2: Weighted 80/20
  Week 3: Weighted 50/50
  Week 4: Weighted 0/100 (fully migrated)
```

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
CNAME at apex	Cannot use CNAME for root domain. Use ALIAS
TTL and failover	If TTL is 300s and failover triggers, clients cache the old IP for 5 min. Set low TTL for critical failover records
Geolocation vs Latency	Geolocation routes by location. Latency routes by measured speed. European user could get US endpoint via latency routing if it's faster
Health check intervals	Standard = 30s. Fast = 10s. From 15 locations = 150 requests/10s to your endpoint
Alias targets	ALIAS can point to ALB, NLB, CloudFront, S3, API Gateway, other Route 53 records — not arbitrary IPs
Private HZ + VPC	Private HZ must be associated with VPC to resolve within it

---

3.6 CloudFront — Distributions, Behaviors, Origins, Cache Invalidation

---

🟢 What It Is in Simple Terms

CloudFront is AWS's CDN (Content Delivery Network). It caches your content at 450+ edge locations around the world so users get responses from a server near them instead of your origin far away. It reduces

latency, reduces load on your origin, and acts as a security layer.

⚙️ How It Works Internally

```
Without CloudFront:
User in Tokyo → API in us-east-1 → ~150ms latency

With CloudFront:
User in Tokyo → Edge in Tokyo → ~5ms latency (if cached)
                              → or fetches from origin once, caches

CloudFront Request Flow:
1. User requests image.jpg
2. DNS resolves to nearest CloudFront edge (via Anycast)
3. Edge checks cache:
   HIT  → Returns cached response. Origin not contacted.
   MISS → Edge fetches from origin, caches, returns to user

Cache Key:
What determines if two requests hit the same cache entry?
Default key = URL path only
Custom key  = URL + specific headers + query strings + cookies

Getting the cache key right is CRITICAL for performance.
Include too much → poor cache hit rate
Include too little → wrong content served to wrong users
```

🧩 Key Components

Distributions

```
A CloudFront Distribution = your CDN configuration

Distribution settings:
├── Domain: d1234abcd.cloudfront.net (or your custom domain)
├── Alternate domain names (CNAMEs): myapp.com, www.myapp.com
├── SSL cert: ACM cert (must be in us-east-1 for CloudFront!)
├── HTTP version: HTTP/2 and HTTP/3 supported
├── IPv6: Enable for modern clients
├── Logging: S3 bucket for access logs
└── Price class: All edges, or subset (Americas+Europe = cheaper)
```

Origins

```
An Origin is where CloudFront fetches content on cache miss.

Origin types:
├── S3 Bucket
│   └── Use Origin Access Control (OAC) to restrict S3 to CF only
│
├── ALB or NLB
│   └── For dynamic content / API
│
├── EC2 instance
│   └── Direct to instance (unusual — use ALB instead)
│
├── Custom HTTP origin (any web server)
│   └── On-prem servers, other cloud providers
│
└── Lambda Function URL
    └── Serverless origin

Origin Shield:
Extra caching layer to reduce origin hits.
Single region concentrates cache — fewer origin requests.
```

Behaviors ⚠ (Critical Concept)

```
A Behavior = routing rules for a URL path pattern.

Each distribution has:
├── Default behavior (* — matches everything)
└── Additional behaviors (evaluated in ORDER, first match wins)

Example: Single CloudFront distribution serving multiple origins

┌──────────────────────────────────────────────────────────┐
│  CloudFront Distribution: myapp.com                      │
│                                                          │
│  Behaviors (in order):                                   │
│  ┌──────────────────┬──────────────────┬──────────────┐  │
│  │ Path Pattern     │ Origin           │ Cache Setting │  │
│  ├──────────────────┼──────────────────┼──────────────┤  │
│  │ /api/*           │ ALB              │ No cache (0s) │  │
│  │ /images/*        │ S3               │ Cache 1 year  │  │
│  │ /static/*        │ S3               │ Cache 1 year  │  │
│  │ * (default)      │ ALB (SPA)        │ Cache 1 hour  │  │
│  └──────────────────┴──────────────────┴──────────────┘  │
└──────────────────────────────────────────────────────────┘

Per-behavior settings:
├── Cache policy (TTL, cache key)
├── Origin request policy (what headers to forward to origin)
```

```
├── Viewer protocol policy (HTTP→HTTPS redirect, HTTPS only)
├── Allowed HTTP methods (GET only, or include POST/PUT)
├── Compress objects automatically
├── Lambda@Edge or CloudFront Functions
└── Trusted key groups (signed URLs/cookies)
```

Cache Policies & TTLs

```
Cache-Control headers from origin:
Cache-Control: max-age=86400   → cache for 1 day
Cache-Control: no-cache        → always revalidate with origin
Cache-Control: no-store        → never cache
Cache-Control: s-maxage=3600   → CDN-specific TTL (overrides max-age)

CloudFront TTL settings (in behavior):
├── Minimum TTL: Floor (even if origin says shorter, CF uses min)
├── Default TTL: Used when origin sends no cache headers
└── Maximum TTL: Ceiling (even if origin says longer, CF uses max)

⚠ Gotcha: If you set Default TTL = 86400 but origin sends
   Cache-Control: max-age=0, CloudFront still caches for DEFAULT TTL
   (until minimum TTL overrides it). This is non-obvious behavior.
```

Cache Invalidation

```
When you need to force CloudFront to refetch content:

Option 1: Invalidation (instant but costs money)
aws cloudfront create-invalidation \
  --distribution-id EDFDVBD6EXAMPLE \
  --paths "/images/logo.png" "/css/*" "/*"

Cost: First 1,000 paths/month free. Then $0.005 per path.
"/*" counts as 1 path — invalidates everything for $0.005.

Option 2: Versioned file names (BEST PRACTICE)
Instead of logo.png → use logo.v2.png or logo.abc123.png
Old version cached until TTL expires (harmless)
New version immediately served from origin
No invalidation needed, no cost.

Option 3: Cache-Control headers
Set short TTL on frequently changing content.
Set long TTL on static assets (with versioned names).

Deployment pattern (immutable assets):
- HTML:   short TTL (5 min)  — changes with each deploy
```

```
- JS/CSS: long TTL (1 year)  + versioned names (main.abc123.js)
- Images: long TTL (1 year)  + versioned names
```

CloudFront Security

```
Origin Access Control (OAC):
Prevents users from bypassing CloudFront and hitting S3 directly.

S3 bucket policy (allows ONLY CloudFront):
{
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Service": "cloudfront.amazonaws.com"
    },
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::my-bucket/*",
    "Condition": {
      "StringEquals": {
        "AWS:SourceArn": "arn:aws:cloudfront::123:distribution/EDFD"
      }
    }
  }]
}

WAF Integration:
Attach AWS WAF to CloudFront distribution.
WAF rules evaluated at edge BEFORE hitting origin.
Block SQLi, XSS, rate limits, geo-blocks globally.

Signed URLs & Signed Cookies:
For private content (paid content, user-specific files):
├── Signed URL:    per-file access with expiry
└── Signed Cookie: access to multiple files (e.g., whole video library)
```

CloudFront Functions vs Lambda@Edge

```
┌──────────────────┬──────────────────┬──────────────────┐
│ Feature          │ CF Functions     │ Lambda@Edge      │
├──────────────────┼──────────────────┼──────────────────┤
│ Runtime          │ JS (subset)      │ Node.js, Python  │
│ Execution time   │ < 1ms            │ up to 30s        │
│ Memory           │ 2MB              │ 128MB-10GB       │
│ Location         │ 218+ edge PoPs   │ 13 regional PoPs │
│ Cost             │ $0.1/1M          │ $0.6/1M          │
│ Network access   │ ❌ No            │ ✅ Yes           │
│ Use case         │ URL rewriting,   │ A/B testing,     │
```

```
│                    │ header manip,     │ auth, body mod,   │
│                    │ simple redirects  │ API calls         │
└────────────────────┴───────────────────┴───────────────────┘
```

💬 Short Crisp Interview Answer

"CloudFront is AWS's CDN with 450+ edge locations globally. A distribution has origins — where content comes from — and behaviors, which are URL-pattern-based routing rules that define what gets cached and how. For a typical SaaS app: static assets go to S3 with 1-year TTL and versioned filenames, API calls go to ALB with cache disabled — all behind a single CloudFront distribution. Use Origin Access Control to lock down S3 so users can't bypass CloudFront. For cache invalidation, versioned file names are better than explicit invalidations — no cost and no propagation delay. The ACM cert for CloudFront must be in us-east-1 regardless of where your origin is — that's a frequent gotcha."

🏭 Real World Production Example

```
React SPA + API on CloudFront:

myapp.com (CloudFront distribution)
│
├── Behavior: /api/* → Origin: ALB
│   Cache: Disabled (no-store)
│   Methods: GET, POST, PUT, DELETE, PATCH
│   Forward: All headers, all cookies, all query strings
│
├── Behavior: /static/* → Origin: S3
│   Cache: 1 year (31536000s)
│   Files: main.abc123.js, vendor.def456.css, logo.v3.png
│   Compress: Yes (gzip/brotli)
│
└── Default: /* → Origin: S3 (index.html)
    Cache: 5 minutes
    Custom error: 404 → index.html (SPA routing)

Security:
- S3 bucket: private, accessible only via OAC
- WAF: Rate limit 1000 req/5min per IP
- HTTPS only: HTTP → HTTPS redirect
- HSTS header: added via CF Function

Deploy process:
1. Build: generates hashed filenames (main.abc123.js)
2. Upload to S3
3. No invalidation needed — new hash = new URL
4. Only index.html changes — invalidate /index.html only
```

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
ACM cert must be us-east-1	CloudFront only accepts ACM certs from us-east-1, even if your origin is elsewhere
Behavior order matters	First matching behavior wins — more specific paths must be ordered before less specific
Cache on dynamic content	Accidentally caching API responses = users see stale or wrong data. Always set no-store on API behaviors
Invalidation propagation	Takes 10-60 seconds to propagate to all edges globally
Query strings in cache key	By default, query strings NOT included in cache key. ?user=123 and ?user=456 hit same cache entry
POST requests not cached	CloudFront forwards POST to origin — never cached
Price Class	"All" includes expensive regions. "Price Class 100" = Americas + Europe only

---

3.7 PrivateLink

🟢 What It Is in Simple Terms

AWS PrivateLink lets you expose a service from your VPC to other VPCs or AWS accounts **privately** — without VPC peering, without internet, without overlapping CIDR concerns. It's a one-way private tunnel using the AWS backbone.

⚙️ How It Works

```
Without PrivateLink (problems):
Consumer VPC wants to use Producer VPC's service.
Options: VPC Peering (full network access — too permissive)
         Internet (public exposure — insecure)

With PrivateLink (purpose-built):
┌──────────────────────┐        ┌──────────────────────┐
│   Consumer VPC       │        │   Producer VPC       │
│   (any CIDR)         │        │   (any CIDR)         │
│                      │        │                      │
│   ┌────────────────┐ │        │   ┌────────────────┐ │
│   │ Interface EP   │ │        │   │ NLB            │ │
│   │ (ENI, priv     │◄────────────│ ← Service      │ │
│   │  IP in VPC)    │ │Priv    │   │   running on   │ │
```

```
│    └──────────────┘    │Link    │    │    EC2/ECS     │  │
│                        │        │    └────────────────┘  │
└────────────────────────┘        └────────────────────────┘

Key properties:
├── No VPC peering needed
├── No overlapping CIDR problem (CIDRs don't matter)
├── Traffic flows only FROM consumer TO service (unidirectional)
├── Consumer gets a private IP in their own VPC
├── NLB required on producer side
└── Works cross-account, cross-region
```

🧩 Use Cases

```
1. SaaS vendor exposing service to customer VPCs
   Example: Datadog, Snowflake, Salesforce all use PrivateLink.
   Customer: creates interface endpoint to vendor's endpoint service.
   Traffic: stays on AWS backbone, never public internet.

2. Internal platform teams exposing services
   Platform team owns "payment-service" → exposes via PrivateLink.
   Consumer teams create interface endpoint in their VPC.
   No peering needed → no broad network access granted.

3. AWS services themselves
   Every "Interface Endpoint" for AWS services IS PrivateLink.
   (SSM, ECR, CloudWatch, etc. are all PrivateLink under the hood)

4. Marketplace products
   AWS Marketplace vendors expose products via PrivateLink.

CIDR overlap is not a concern with PrivateLink — this is one of its key
advantages over VPC Peering. The endpoint receives an IP from the
consumer's own CIDR space; no routing to the producer's CIDR is involved.

Always set acceptance-required=true for cross-account production services
— without it, any account knowing the service name can connect without
approval.
```

```
# Producer: Create endpoint service from NLB
aws ec2 create-vpc-endpoint-service-configuration \
  --network-load-balancer-arns arn:aws:elasticloadbalancing:... \
  --acceptance-required   # true = manually approve consumers

# Consumer: Create interface endpoint to service
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-consumer \
  --vpc-endpoint-type Interface \
```

```
    --service-name com.amazonaws.vpce.us-east-1.vpce-svc-0abc123 \
    --subnet-ids subnet-123 \
    --security-group-ids sg-456
```

💬 Short Crisp Interview Answer

"PrivateLink lets you expose a service from one VPC to other VPCs or accounts privately, without VPC peering. The service producer puts a Network Load Balancer in front of their service and registers it as an endpoint service. Consumers create an Interface Endpoint in their VPC, which gives them a private ENI with an IP in their own CIDR. Traffic flows one-way from consumer to service, staying entirely on the AWS backbone. The key benefit over VPC peering is granularity — consumers can only reach the specific service, not the entire VPC network. This is how all AWS managed services work internally, and how SaaS vendors like Datadog and Snowflake provide private connectivity."

⚠️ Tricky Edge Cases / Gotchas

Gotcha	Detail
NLB required	Must use NLB (not ALB) on producer side
One-way only	Consumer can reach producer's service — not the other way around
Acceptance required	Producer can require manual approval of each consumer connection
DNS	Consumer needs Route 53 private hosted zone or use auto-generated DNS name
Cross-region PrivateLink	Possible via Interface Endpoint + cross-region NLB, but complex

---

3.8 Direct Connect & Site-to-Site VPN

🟢 What It Is in Simple Terms

Both connect your on-premises data center to AWS. VPN uses the public internet (encrypted). Direct Connect uses a dedicated physical fiber cable — private, consistent bandwidth, no internet.

⚙️ Site-to-Site VPN

```
Architecture:
On-Prem Data Center ←──IPSec VPN tunnel──→ AWS VPC

Components:
├── Virtual Private Gateway (VGW) → AWS side of VPN
```

```
│   Attached to your VPC
│
├── Customer Gateway (CGW) → Represents your on-prem router
│   AWS resource that holds your router's public IP + BGP ASN
│
└── VPN Connection → Links VGW and CGW
    Two tunnels (HA): active/passive by default

Traffic flow:
On-prem server → on-prem router → IPSec tunnel
→ VGW → VPC route table → EC2

Specs:
├── Setup time: hours (software config)
├── Bandwidth: ~1.25 Gbps per tunnel (2 tunnels = 2.5 Gbps)
├── Latency: variable (goes over public internet)
├── Cost: ~$0.05/hr + data transfer
└── Encryption: AES-256, IKEv1/IKEv2

Route propagation:
Enable "propagate routes" on VPC route table.
On-prem CIDRs automatically appear in VPC route table.
```

```
# Create Customer Gateway (your on-prem router)
aws ec2 create-customer-gateway \
  --type ipsec.1 \
  --public-ip 203.0.113.1 \   # your router's public IP
  --bgp-asn 65000

# Create Virtual Private Gateway
aws ec2 create-vpn-gateway --type ipsec.1
aws ec2 attach-vpn-gateway \
  --vpn-gateway-id vgw-0abc123 \
  --vpc-id vpc-0abc123

# Create VPN connection
aws ec2 create-vpn-connection \
  --type ipsec.1 \
  --customer-gateway-id cgw-0abc123 \
  --vpn-gateway-id vgw-0abc123
```

⚙️ Direct Connect (DX)

```
Architecture:
On-Prem ──physical fiber──→ AWS DX Location ──→ AWS Region

Components:
├── DX Location: Co-location facility where AWS has a presence
```

```
  ├── Cross-connect: Physical cable from your router to AWS DX router
  ├── Virtual Interface (VIF):
  │   ├── Private VIF → connects to VPC via VGW
  │   ├── Public VIF  → connects to AWS public services (S3, etc.)
  │   └── Transit VIF → connects to Transit Gateway
  └── DX Connection: The physical 1G, 10G, or 100G port

  DX Setup options:
  ├── Dedicated Connection: 1G, 10G, 100G — AWS provides physical port.
  │   Order directly from AWS. Takes weeks to provision.
  │
  └── Hosted Connection: < 1G or fractional bandwidth.
      AWS Direct Connect partner provisions and shares their port.
      You buy a slice of their connection.

  Specs:
  ├── Setup time: weeks to months (physical cabling)
  ├── Bandwidth: 1Gbps to 100Gbps
  ├── Latency: consistent, predictable (no internet jitter)
  ├── NOT encrypted by default (private line, not encrypted)
  └── Cost: port cost + data transfer (expensive)

  Redundancy pattern (critical):
  Single DX = single point of failure.

  HA options:
  ┌──────────────────────────────────────────────────────────┐
  │ Option 1: Two DX connections from same location          │
  │           Protects against port failure only             │
  │                                                          │
  │ Option 2: DX + VPN backup (recommended for most)         │
  │           DX primary → VPN failover if DX fails          │
  │           Different physical paths                       │
  │                                                          │
  │ Option 3: Two DX from different DX locations             │
  │           Maximum resilience (protects against           │
  │           entire DX location failure)                    │
  └──────────────────────────────────────────────────────────┘

  DX + Encryption (if required):
  Direct Connect is NOT encrypted.
  Run IPSec VPN over the DX connection for encryption.
  (Public VIF → VPN → private connectivity + encryption)
```

DX vs VPN — Comparison

```
  ┌──────────────────┬──────────────────────┬──────────────────────────┐
  │ Feature          │ Site-to-Site VPN     │ Direct Connect           │
  ├──────────────────┼──────────────────────┼──────────────────────────┤
  │ Setup time       │ Hours                │ Weeks to months          │
```

```
│ Bandwidth          │ 1.25 Gbps/tunnel     │ 1-100 Gbps            │
│ Latency            │ Variable (internet)  │ Consistent, low       │
│ Path               │ Public internet      │ Private fiber         │
│ Encryption         │ ✅ Built-in (IPSec)  │ ❌ Not by default     │
│ Cost               │ ~$0.05/hr + transfer │ Port + transfer       │
│ HA                 │ 2 tunnels built-in   │ Need redundant links  │
│ Use case           │ Backup, small BW,    │ High BW, compliance,  │
│                    │ quick setup          │ consistent latency    │
└────────────────────┴──────────────────────┴───────────────────────┘

Interview answer: "Which do you use?"
→ Start with VPN (fast, cheap, encrypted)
→ Upgrade to DX when BW > 1Gbps, need consistent latency,
  or compliance requires private connectivity
→ Keep VPN as DX backup
```

💬 Short Crisp Interview Answer

"Both connect on-premises to AWS. Site-to-Site VPN uses IPSec over the public internet — setup takes hours, bandwidth is ~1.25 Gbps per tunnel, it's encrypted by default, and it's cheap. Direct Connect is a dedicated private fiber connection through an AWS colocation facility — setup takes weeks, bandwidth scales to 100 Gbps, latency is consistent, but it's not encrypted by default and costs significantly more. In production, the common pattern is Direct Connect as primary for high-bandwidth, latency-sensitive workloads, with a VPN as backup path. If you need both private connectivity AND encryption over DX, you run an IPSec VPN over the DX connection."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
DX is not encrypted	Physical private line but no encryption. Add VPN over DX for encryption
VPN over DX	Uses Public VIF + VPN — gives both private path AND encryption
Single DX = SPOF	Always plan redundancy: second DX or DX + VPN backup
BGP required for DX	Must configure BGP on your router for DX routing
DX failover to VPN	Not automatic — need to configure BGP routing priorities
Hosted vs Dedicated DX	Hosted: faster setup, fractional bandwidth. Dedicated: full port, longer provisioning

- **BFD is mandatory for fast DX failover** — without it, BGP hold-down timer (default 90s) means 90-second failover. BFD reduces detection to ~900ms. Always enable BFD on DX Virtual Interfaces in production.
- **TCP sessions survive a 3-second outage** — TCP retransmission timeout is 1-3 minutes. A brief BGP reconvergence window does not drop established sessions. Candidates often over-engineer this concern.

- **Graceful fail-back means never forcing sessions off VPN** — allow existing VPN sessions to terminate naturally. Forcing a cut-over drops active sessions and defeats the purpose of graceful failover.

3.9 Network ACLs Deep Dive & Stateless vs Stateful

🟢 What It Is in Simple Terms

A deeper look at why statelessness makes NACLs fundamentally different to work with, and how rule ordering can silently break or unexpectedly allow traffic.

⚙️ Stateless Deep Dive — Why It Matters

```
Stateless = the firewall has NO MEMORY of previous packets.
Each packet evaluated independently, both directions.

TCP connection example (stateless problem):
Step 1: Client (1.2.3.4:54321) → Server (:443)  [SYN]
Step 2: Server → Client (1.2.3.4:54321)          [SYN-ACK]
Step 3: Client → Server                          [ACK]

For stateless NACL:
INBOUND rule needed:  allow TCP from 0.0.0.0/0 port 443
OUTBOUND rule needed: allow TCP to 0.0.0.0/0 port 1024-65535
                      (ephemeral port range — the response port)

If OUTBOUND ephemeral rule is missing:
→ SYN arrives (inbound 443 ✅ allowed)
→ SYN-ACK sent (outbound 54321 ❌ BLOCKED by NACL)
→ Connection silently fails
→ No error — just timeout
→ Very hard to debug!

Ephemeral port ranges by OS:
Linux:   32768-60999
Windows: 49152-65535
NAT GW:  1024-65535 (AWS uses full range)
Recommendation: Allow 1024-65535 outbound to cover all cases.
```

⚙️ NACL Rule Evaluation Deep Dive

```
Rules evaluated in ASCENDING numerical order.
First match = action taken. Processing STOPS.
```

```
Example:
Rule 50:  DENY  192.168.1.0/24
Rule 100: ALLOW 0.0.0.0/0
Rule *:   DENY  all (implicit, always last)

Request from 192.168.1.5:
→ Check rule 50: matches 192.168.1.0/24 → DENY. STOP.
→ Rule 100 never evaluated.

Request from 10.0.0.5:
→ Check rule 50: no match
→ Check rule 100: matches 0.0.0.0/0 → ALLOW. STOP.

Numbering best practice:
Use increments of 10 or 100.
Leave gaps for future rules.
Rule 100, 110, 120... (can insert 105 without renumbering)

⚠ Critical gotcha: NACL deny rules are evaluated in ORDER.
If you add a DENY for a bad IP, put it at a LOW rule number
(e.g., rule 50) so it fires BEFORE the allow all (rule 100).
If you add DENY at rule 200 and ALLOW all is at rule 100,
the deny NEVER fires — traffic is already allowed!
```

⚙️ Stateful vs Stateless — Complete Comparison

```
Security Group (STATEFUL):
Request:  Client → Server port 443   ← allowed by inbound rule
Response: Server → Client port 54321 ← AUTO ALLOWED (connection tracked)

NACL (STATELESS):
Request:  Client → Server port 443   ← evaluated against inbound rules
Response: Server → Client port 54321 ← evaluated against OUTBOUND rules
                                        (separate, independent check)

Connection tracking in Security Groups:
SG tracks the connection 5-tuple:
(source IP, source port, dest IP, dest port, protocol)

Even if you REMOVE the inbound rule after a connection is established,
existing connections continue until they close.
New connections after rule change = blocked.
Existing connections before rule change = continue.
```

💬 Short Crisp Interview Answer

"The fundamental difference is statefulness. Security Groups are stateful — they track connections, so return traffic is automatically allowed without an explicit outbound rule. NACLs are stateless — every packet is evaluated independently, so you must allow both inbound request traffic AND outbound response traffic, including the ephemeral port range 1024-65535. NACL rules also evaluate in numerical order and stop at the first match, so rule ordering is critical — a deny rule must have a lower number than any allow rule that would cover the same traffic. The most common NACL mistake is forgetting outbound ephemeral ports, which causes silent connection timeouts that are difficult to debug."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Missing ephemeral ports	Most common NACL mistake. Causes silent timeouts. Always allow 1024-65535 outbound
Deny rule number order	Deny at rule 200, Allow-all at rule 100 = deny never fires
Existing connections and SG changes	Removing an SG rule doesn't kill existing connections immediately
NACL vs SG for same-VPC traffic	SGs don't implicitly allow between instances in same VPC. NACLs apply to cross-subnet traffic too

---

3.10 Egress-Only IGW, NAT Gateway vs NAT Instance

🟢 Egress-Only Internet Gateway

```
Purpose: For IPv6-only outbound internet access from private resources.

Why it exists:
IPv4 private addresses (10.x, 172.16.x, 192.168.x) are not
routable on the internet — NAT translates them.

IPv6 addresses are ALL globally unique and routable.
If you put an IPv6 address on an instance and add a route to IGW,
it's reachable FROM the internet! No NAT needed.

But what if you want IPv6 instances to reach the internet
WITHOUT being reachable inbound?

→ Egress-Only Internet Gateway!

Egress-Only IGW:
├── Allows outbound IPv6 traffic to internet
├── Blocks inbound IPv6 connections from internet
├── No charge
└── Conceptually like a NAT Gateway but for IPv6
```

```
Route table for IPv6 private subnet:

┌──────────────────────┬──────────────────────────────┐
│ ::/0                 │ eigw-0abc123 (Egress-Only IGW)│
│ 2001:db8::/32        │ local                        │
└──────────────────────┴──────────────────────────────┘
```

🔧 NAT Gateway vs NAT Instance

```
┌──────────────────────┬──────────────────────┬──────────────────────┐
│ Feature              │ NAT Gateway          │ NAT Instance         │
├──────────────────────┼──────────────────────┼──────────────────────┤
│ Type                 │ Managed AWS service  │ EC2 instance         │
│ Availability         │ Highly available     │ Single EC2 (SPOF)    │
│                      │ (within AZ)          │ unless you add ASG   │
│ Bandwidth            │ Scales to 45 Gbps    │ Depends on EC2 type  │
│ Maintenance          │ AWS managed          │ You patch/maintain   │
│ Security groups      │ Not applicable       │ Required             │
│ Bastion host         │ ❌ Cannot use as     │ ✅ Can double as     │
│                      │ bastion              │ bastion/jump host    │
│ Port forwarding      │ ❌ Not supported     │ ✅ Supported         │
│ Cost                 │ $0.045/hr + $0.045/GB│ EC2 cost only        │
│ Disable src/dst chk  │ Not needed           │ MUST disable!        │
│ Performance          │ Very high            │ Limited by inst type │
└──────────────────────┴──────────────────────┴──────────────────────┘

When would you use NAT Instance?
├── Very cost-sensitive, low-traffic environments
├── Need port forwarding
├── Need it to also serve as a bastion host
└── Legacy environments

For everything else: NAT Gateway.

⚠ Critical NAT Instance setting:
Must DISABLE source/destination check on the EC2 instance.
By default, EC2 drops packets where it is neither source nor dest.
NAT = forwards others' packets → must disable this check.

aws ec2 modify-instance-attribute \
  --instance-id i-nat-instance \
  --no-source-dest-check
```

💬 Short Crisp Interview Answer

"Egress-Only Internet Gateway is for IPv6 — it allows IPv6 instances in private subnets to initiate outbound connections to the internet but blocks inbound connections, similar to how NAT Gateway works for IPv4. On NAT Gateway versus NAT Instance: NAT Gateway is the managed, scalable, highly available option — scales

to 45 Gbps, AWS handles everything. NAT Instance is a DIY EC2-based NAT — cheaper but you manage HA, patching, and scaling. The critical gotcha on NAT Instance is that you must disable the source/destination check, otherwise EC2 drops forwarded packets. In almost all modern architectures, NAT Gateway is the right choice."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Egress-Only IGW is IPv6 only	Cannot use for IPv4 traffic
NAT GW AZ-bound	Needs one per AZ for HA. Cross-AZ NAT traffic = extra cost + SPOF
NAT Instance src/dest check	Must disable or NAT won't work
IPv6 + regular IGW	IPv6 on instance + route to IGW = publicly reachable! Use Egress-Only for private IPv6
NAT GW does not support IPv6	For IPv6 private outbound, Egress-Only IGW is the answer, not NAT GW

🔗 Category 3 — Full Connections Map

```
NETWORKING connects to:

VPC
├── EC2/ECS/EKS/RDS    → Resources living in subnets
├── Security Groups    → Instance-level stateful firewall
├── NACLs              → Subnet-level stateless firewall
├── Route Tables       → Traffic routing decisions
├── IGW                → Internet access for public subnets
├── NAT Gateway        → Outbound internet for private subnets
├── VPC Peering        → Direct VPC-to-VPC connectivity
├── Transit Gateway    → Hub-and-spoke multi-VPC routing
├── VPC Endpoints      → Private access to AWS services
│   ├── Gateway EP     → S3, DynamoDB (free, route-based)
│   └── Interface EP   → All services (PrivateLink-based)
├── PrivateLink        → Expose services to other VPCs privately
├── Route 53           → DNS (public + private hosted zones)
├── CloudFront         → CDN, global edge caching, security
├── Direct Connect     → Dedicated private connection to on-prem
├── Site-to-Site VPN   → Encrypted tunnel to on-prem over internet
├── Flow Logs          → VPC traffic logging to S3/CloudWatch
├── Egress-Only IGW    → IPv6 private outbound only
└── WAF/Shield         → Edge protection (integrates with CF/ALB)
```

📌 Quick Reference — Interview Cheat Sheet

Topic	Key Fact
VPC CIDR	Immutable after creation. Min /28, max /16
Reserved IPs per subnet	5 (network, router, DNS, future, broadcast)
Public subnet definition	Subnet with route to IGW (routing concept, not a flag)
NAT GW HA	One per AZ. Each private subnet routes to local NAT GW
NAT GW bandwidth	Scales to 45 Gbps automatically
SG = stateful	Return traffic auto-allowed
NACL = stateless	Must allow ephemeral ports (1024-65535) both directions
NACL rule evaluation	Lowest number first. First match wins. Stop processing
VPC Peering transitivity	Non-transitive. A→B, B→C ≠ A→C
TGW vs Peering	≤3 VPCs = Peering (free). >3 or on-prem = TGW
Gateway Endpoint	Free. S3 and DynamoDB only. Route-based
Interface Endpoint	Paid. All services. ENI-based. Works from on-prem
Route 53 ALIAS	Use at zone apex. Free for AWS resource targets
CNAME at apex	Not allowed. Use ALIAS instead
CloudFront ACM cert	Must be in us-east-1
DX encryption	Not encrypted by default. Add VPN over DX if needed
NAT Instance gotcha	Must disable source/destination check
Egress-Only IGW	IPv6 outbound only. Not for IPv4
PrivateLink NLB	NLB required on producer side (not ALB)
NACL deny rule tip	Put deny at LOW rule number (e.g., 50) before allow-all (100)
VPN bandwidth	~1.25 Gbps per tunnel. 2 tunnels = 2.5 Gbps max
Direct Connect bandwidth	1 Gbps to 100 Gbps dedicated port
Behavior order in CloudFront	First matching path pattern wins
Peering route tables	Must manually add routes on BOTH sides

Category 3: Networking & VPC — Complete Interview Guide
Excluded topics: CodeBuild, CodePipeline, CloudFormation